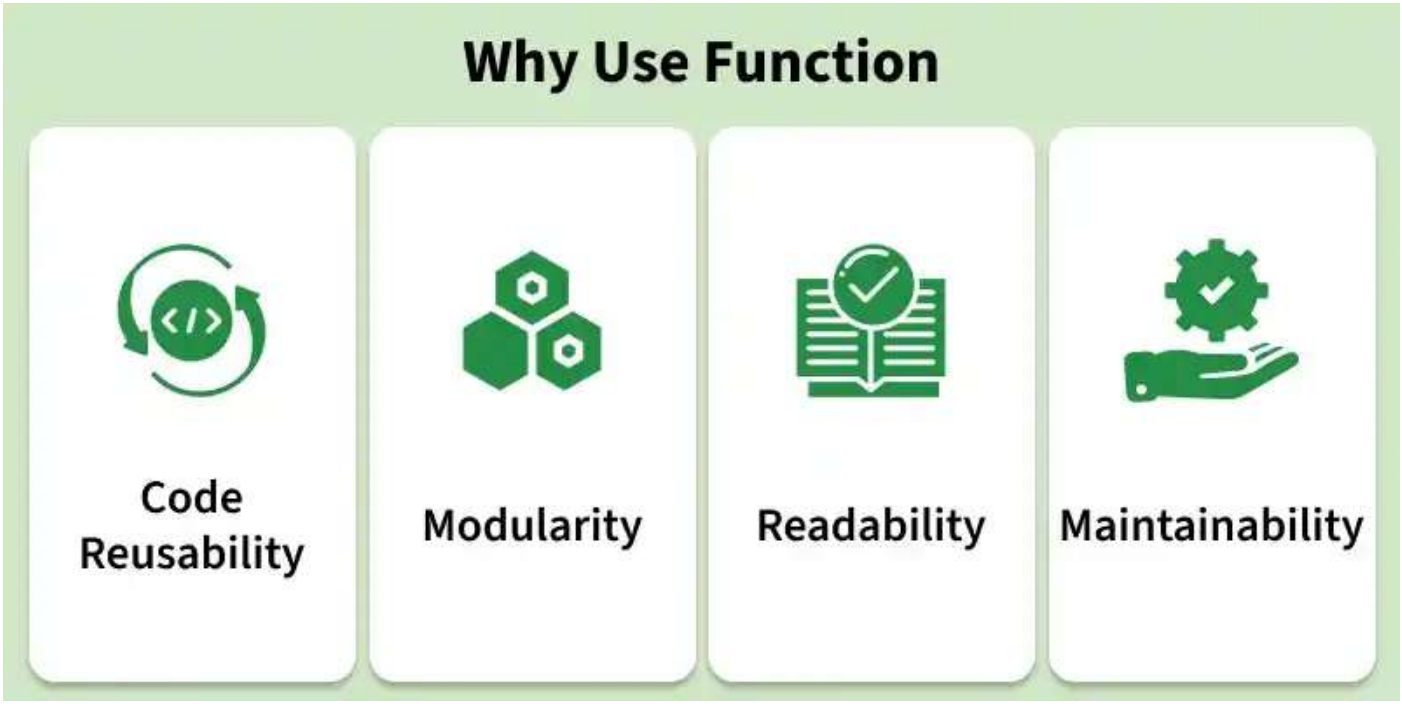
Python Functions

Last Updated : 19 Sep, 2026

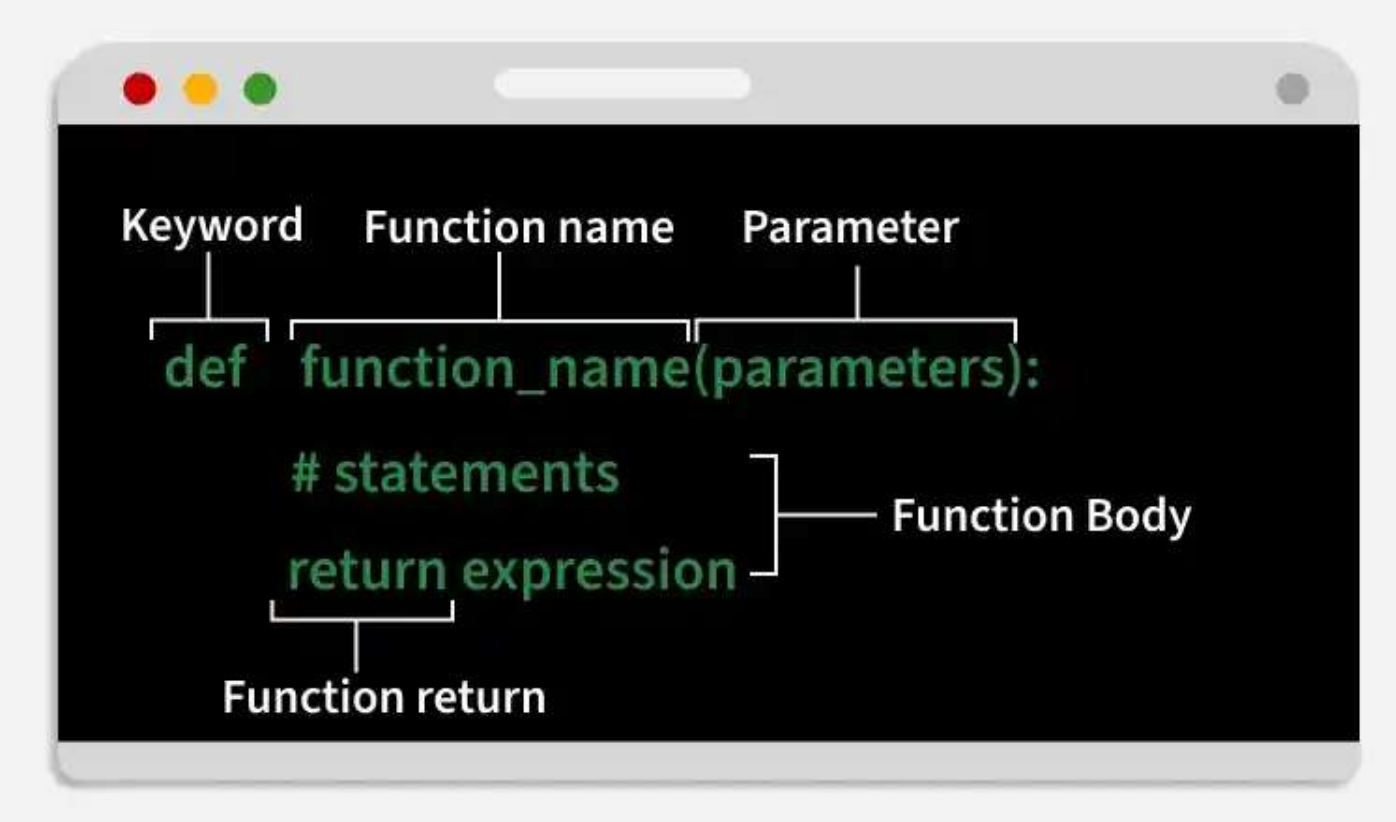
Python functions are reusable blocks of code used to perform a specific task. They help organize programs into smaller sections and execute the same logic whenever needed by calling the function.



Features of a function

Defining a Function

A function can be defined using def keyword. Below is the syntax to define a function:



Syntax of Python Function Declaration

Here, we define a function using def that prints a welcome message when called.

```
def fun():
    print("Welcome to GFG")
```

No Output

Calling a Function

After creating a function, call it by using the name of the functions followed by parenthesis containing parameters of that particular function.

```
def fun():
    print("Welcome to GFG")

fun()
```

Welcome to GFG

Output

```
Welcome to GFG
```

We can also assign a Function to a variable, and call that function using that variable.

Function Arguments

Arguments are values passed to a function when it is called. They allow functions to receive input data and perform operations using those values.

Syntax

```
def function_name(arguments):
    # function body
    return value
```

- def function_name(arguments): defines a function with optional arguments.
- # function body contains the statements to be executed.
- return value returns a result from the function. If no return statement is used, it returns None by default.

Example: In this example, function checks whether the number passed as an argument is even or odd.

```
def evenOdd(x):
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"

print(evenOdd(16))
print(evenOdd(7))
```

```
Even
Odd
```

Output

```
Even
Odd
```

Types of Function Arguments

Python supports different types of arguments that can be passed during a function call.

1. Default argument: Default argument use a predefined value when no value is passed during the function call.

```
def myFun(x, y=50):
    print("x: ", x)
    print("y: ", y)

myFun(10)
```

```
x:  10
y:  50
```

Output

```
x:  10
```

```
y:  50
```

Explanation:

- y=50 sets a default value for parameter y and myFun(10) passes only one argument.
- Since y is not provided, it uses the default value 50.

2. Keyword Arguments: pass values using parameter names, so argument order does not matter.

```
def student(fname, lname):
    print(fname, lname)

student(fname='Geeks', lname='Practice')
student(lname='Practice', fname='Geeks')
```

```
Geeks Practice
Geeks Practice
```

Output

```
Geeks Practice
Geeks Practice
```

Explanation: fname and lname are passed using parameter names and arguments can be provided in any order.

3. Positional Arguments: values are assigned to parameters based on their order in the function call.

```
def nameAge(name, age):
    print("Hi, I am", name)
    print("My age is ", age)

print("Case-1:")
nameAge("Olivia", 27)

print("Case-2:")
nameAge(27, "Olivia")
```

```
Case-1:
Hi, I am Olivia
My age is  27
Case-2:
Hi, I am 27
My age is  Olivia
```

Output

```
Case-1:
Hi, I am Olivia
My age is  27
Case-2:
```

```
Hi, I am 27
My age is  Olivia
```

Explanation:

- In Case-1, values match the correct parameters.
- In Case-2, values are swapped because the order changed.

4. Arbitrary Arguments: allow functions to accept multiple values. This is done using two special symbols:

- *args collects extra positional arguments as a tuple.
- **kwargs collects extra keyword arguments as a dictionary.

```
def myFun(*args, **kwargs):
    print("Non-Keyword Arguments (*args):")
    for arg in args:
        print(arg)

    print("Keyword Arguments (**kwargs):")
    for key, value in kwargs.items():
        print(f"{key} == {value}")

myFun('Hey', 'Welcome', first='Geeks', mid='for', last='Geeks')
```

Output

```
Non-Keyword Arguments (*args):
Hey
Welcome
Keyword Arguments (**kwargs):
first == Geeks
mid == for
last == Geeks
```

Explanation:

- *args stores extra positional arguments.
- **kwargs stores extra keyword arguments.
- loop prints all positional and keyword values separately.

Function within Functions

A function defined inside another function is called an inner function (or nested function). It is used to organize related logic and access variables from the outer function.

```
def f1():
```

```
    s = 'I love GeeksforGeeks'
    def f2():
        print(s)

    f2()
f1()
```

```
I love GeeksforGeeks
```

Output

```
I love GeeksforGeeks
```

Return Statement

Return is used to end a function and send a value back to the caller. It can return any data type, multiple values (packed into a tuple), or None if no value is given.

Syntax

return [expression]

Parameters: expression is the value returned by the function. If no value is returned, it returns None by default.

```
def sq_value(num):
    return num**2

print(sq_value(2))
print(sq_value(-4))
```

```
4
16
```

Output

```
4
16
```

Explanation:

- sq_value(num) accepts a number as input.
- num**2 calculates the square of the number.
- return sends the result back to the caller.

Pass by Reference and Pass by Value

Variables refer to objects. Function behavior depends on whether the object is mutable or immutable.

- Mutable objects like lists can be modified inside functions.
- Immutable objects like integers and strings remain unchanged.

```
def myFun(x):
    x[0] = 20

b = [10, 11, 12, 13]
myFun(b)
print(b)

def myFun2(x):
    x = 20

a = 10
myFun2(a)
print(a)
```

```
[20, 11, 12, 13]
10
```

Output

```
[20, 11, 12, 13]
10
```

Explanation:

- myFun(x) modifies the first element of the list and lists are mutable, so the original list changes.
- myFun2(x) assigns a new value to x. Integers are immutable, so the original value of a remains unchanged.

> ***Note**: Python uses pass-by-object-reference, where functions receive references to objects instead of actual copies.*



Python Functions



How Functions Work in Python



Applications of Functions.

Python Functions

Visit Course

Suggested Quiz

Which type of argument uses a predefined value when no value is passed during a function call?

A Positional Argument

B Keyword Argument

C Default Argument

D Arbitrary Argument

Report Issue View Explanation

1/5 < Previous Next >

Comment P Pranat... 533

Explore

Python Fundamentals

Python Data Structures

Advanced Python

Data Science with Python

Web Development with Python

Python Practice

Courses

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us

Explore
POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science

Courses
ML and Data Science
DSA and Placements
Web Development

Videos
DSA
Python
Java
C++
Web Development
Data Science

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GFG
Corporate
Solution
Training
Program

DevOps
CS Core
Subjects
Interview
Preparation
Software and
Tools

Programming
Languages
DevOps &
Cloud
GATE
Trending
Technologies

CS Subjects



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved